

续表

名字	头文件
swap	<utility>
terminate	<exception>
time	<ctime>
tolower	<cctype>
toupper	<cctype>
transform	<algorithm>
tuple	<tuple>
tuple_element	<tuple>
tuple_size	<tuple>
type_info	<typeinfo>
unexpected	<exception>
uniform_int_distribution	<random>
uniform_real_distribution	<random>
uninitialized_copy	<memory>
uninitialized_fill	<memory>
unique	<algorithm>
unique_copy	<algorithm>
unique_ptr	<memory>
unitbuf	<iostream>
unordered_map	<unordered_map>
unordered_multimap	<unordered_map>
unordered_multiset	<unordered_set>
unordered_set	<unordered_set>
upper_bound	<algorithm>
uppercase	<iostream>
vector	<vector>
weak_ptr	<memory>

870

A.2 算法概览

标准库定义了超过 100 个算法。要想高效使用这些算法需要了解它们的结构而不是单纯记忆每个算法的细节。因此，我们在第 10 章中关注标准库算法架构的描述和理解。在本节中，我们将简要描述每个算法，在下面的描述中，

- beg 和 end 是表示元素范围的迭代器（参见 9.2.1 节，第 296 页）。几乎所有算法都对一个由 beg 和 end 表示的序列进行操作。
- beg2 是表示第二个输入序列开始位置的迭代器。end2 表示第二个序列的末尾位置（如果有的话）。如果没有 end2，则假定 beg2 表示的序列与 beg 和 end 表示的序列一样大。beg 和 beg2 的类型不必匹配，但是，必须保证对两个序列中的元素都可以执行特定操作或调用给定的可调用对象。
- dest 是表示目的序列的迭代器。对于给定输入序列，算法需要生成多少元素，目的序列必须保证能保存同样多的元素。

- unaryPred 和 binaryPred 是一元和二元谓词（参见 10.3.1 节，第 344 页），分别接受一个和两个参数，都是来自输入序列的元素，两个谓词都返回可用作条件的类型。
- comp 是一个二元谓词，满足关联容器中对关键字序的要求（参见 11.2.2 节，第 378 页）。
- unaryOp 和 binaryOp 是可调对象（参见 10.3.2 节，第 346 页），可分别使用来自输入序列的一个和两个实参来调用。

A.2.1 查找对象的算法

这些算法在一个输入序列中搜索一个指定值或一个值的序列。

每个算法都提供两个重载的版本，第一个版本使用底层类型的相等运算符(==)来比较元素；第二个版本使用用户给定的 unaryPred 和 binaryPred 比较元素。

简单查找算法

这些算法查找指定值，要求输入迭代器（input iterator）。

```
find(beg, end, val)
find_if(beg, end, unaryPred)
find_if_not(beg, end, unaryPred)
count(beg, end, val)
count_if(beg, end, unaryPred)
```

find 返回一个迭代器，指向输入序列中第一个等于 val 的元素。

find_if 返回一个迭代器，指向第一个满足 unaryPred 的元素。

find_if_not 返回一个迭代器，指向第一个令 unaryPred 为 false 的元素。上述三个算法在未找到元素时都返回 end。

count 返回一个计数器，指出 val 出现了多少次；count_if 统计有多少个元素满足 unaryPred。

```
all_of(beg, end, unaryPred)
any_of(beg, end, unaryPred)
none_of(beg, end, unaryPred)
```

这些算法都返回一个 bool 值，分别指出 unaryPred 是否对所有元素都成功、对任意一个元素成功以及对所有元素都不成功。如果序列为空，any_of 返回 false，而 all_of 和 none_of 返回 true。

查找重复值的算法

下面这些算法要求前向迭代器（forward iterator），在输入序列中查找重复元素。

```
adjacent_find(beg, end)
adjacent_find(beg, end, binaryPred)
```

返回指向第一对相邻重复元素的迭代器。如果序列中无相邻重复元素，则返回 end。

```
search_n(beg, end, count, val)
search_n(beg, end, count, val, binaryPred)
```

返回一个迭代器，从此位置开始有 count 个相等元素。如果序列中不存在这样的子序列，

则返回 `end`。

872 查找子序列的算法

在下面的算法中，除了 `find_first_of` 之外，都要求两个前向迭代器。`find_first_of` 用输入迭代器表示第一个序列，用前向迭代器表示第二个序列。这些算法搜索子序列而不是单个元素。

```
search(beg1, end1, beg2, end2)
search(beg1, end1, beg2, end2, binaryPred)
```

返回第二个输入范围（子序列）在第一个输入范围中第一次出现的位置。如果未找到子序列，则返回 `end1`。

```
find_first_of(beg1, end1, beg2, end2)
find_first_of(beg1, end1, beg2, end2, binaryPred)
```

返回一个迭代器，指向第二个输入范围中任意元素在第一个范围中首次出现的位置。如果未找到匹配元素，则返回 `end1`。

```
find_end(beg1, end1, beg2, end2)
find_end(beg1, end1, beg2, end2, binaryPred)
```

类似 `search`，但返回的是最后一次出现的位置。如果第二个输入范围为空，或者在第一个输入范围中未找到它，则返回 `end1`。

A.2.2 其他只读算法

这些算法要求前两个实参都是输入迭代器。

`equal` 和 `mismatch` 算法还接受一个额外的输入迭代器，表示第二个范围的开始位置。这两个算法都提供两个重载的版本。第一个版本使用底层类型的相等运算符（`==`）比较元素，第二个版本则用用户指定的 `unaryPred` 或 `binaryPred` 比较元素。

```
for_each(beg, end, unaryOp)
```

对输入序列中的每个元素应用可调用对象（参见 10.3.2 节，第 346 页）`unaryOp`。`unaryOp` 的返回值（如果有的话）被忽略。如果迭代器允许通过解引用运算符向序列中的元素写入值，则 `unaryOp` 可能修改元素。

```
mismatch(beg1, end1, beg2)
mismatch(beg1, end1, beg2, binaryPred)
```

比较两个序列中的元素。返回一个迭代器的 `pair`（参见 11.2.3 节，第 379 页），表示两个序列中第一个不匹配的元素。如果所有元素都匹配，则返回的 `pair` 中第一个迭代器为 `end1`，第二个迭代器指向 `beg2` 中偏移量等于第一个序列长度的位置。

```
equal(beg1, end1, beg2)
equal(beg1, end1, beg2, binaryPred)
```

确定两个序列是否相等。如果输入序列中每个元素都与从 `beg2` 开始的序列中对应元素相等，则返回 `true`。

873 A.2.3 二分搜索算法

这些算法都要求前向迭代器，但这些算法都经过了优化，如果我们提供随机访问迭代

器（random-access iterator）的话，它们的性能会好得多。从技术上讲，无论我们提供什么类型的迭代器，这些算法都会执行对数次的比较操作。但是，当使用前向迭代器时，这些算法必须花费线性次数的迭代器操作来移动到序列中要比较的元素。

这些算法要求序列中的元素已经是有序的。它们的行为类似关联容器的同名成员（参见 11.3.5 节，第 389 页）。`equal_range`、`lower_bound` 和 `upper_bound` 算法返回迭代器，指向给定元素在序列中的正确插入位置——插入后还能保持有序。如果给定元素比序列中的所有元素都大，则会返回尾后迭代器。

每个算法都提供两个版本：第一个版本用元素类型的小于运算符（<）来检测元素；第二个版本则使用给定的比较操作。在下列算法中，“x 小于 y”表示 `x < y` 或 `comp(x, y)` 成功。

```
lower_bound(beg, end, val)
lower_bound(beg, end, val, comp)
```

返回一个迭代器，表示第一个小于等于 `val` 的元素，如果不存在这样的元素，则返回 `end`。

```
upper_bound(beg, end, val)
upper_bound(beg, end, val, comp)
```

返回一个迭代器，表示第一个大于 `val` 的元素，如果不存在这样的元素，则返回 `end`。

```
equal_range(beg, end, val)
equal_range(beg, end, val, comp)
```

返回一个 pair（参见 11.2.3 节，第 379 页），其 `first` 成员是 `lower_bound` 返回的迭代器，`second` 成员是 `upper_bound` 返回的迭代器。

```
binary_search(beg, end, val)
binary_search(beg, end, val, comp)
```

返回一个 `bool` 值，指出序列中是否包含等于 `val` 的元素。对于两个值 `x` 和 `y`，当 `x` 不小于 `y` 且 `y` 也不小于 `x` 时，认为它们相等。

A.2.4 写容器元素的算法

很多算法向给定序列中的元素写入新值。这些算法可以从不同角度加以区分：通过表示输入序列的迭代器类型来区分；或者通过是写入输入序列中元素还是写入给定目的位置来区分。

只写不读元素的算法

这些算法要求一个输出迭代器（output iterator），表示目的位置。`_n` 结尾的版本接受第二个实参，表示写入的元素数目，并将给定数目的元素写入到目的位置中。

```
fill(beg, end, val)
fill_n(dest, cnt, val)
generate(beg, end, Gen)
generate_n(dest, cnt, Gen)
```

给输入序列中每个元素赋予一个新值。`fill` 将值 `val` 赋予元素；`generate` 执行生成器对象 `Gen()` 生成新值。生成器是一个可调用对象（参见 10.3.2 节，第 346 页），每次调用会生成一个不同的返回值。`fill` 和 `generate` 都返回 `void`。`_n` 版本返回一个迭代器，指向写入到输出序列的最后一个元素之后的位置。

使用输入迭代器的写算法

这些算法读取一个输入序列，将值写入到一个输出序列中。它们要求一个名为 `dest` 的输出迭代器，而表示输入范围的迭代器必须是输入迭代器。

```
copy(beg, end, dest)
copy_if(beg, end, dest, unaryPred)
copy_n(beg, n, dest)
```

从输入范围将元素拷贝到 `dest` 指定的目的序列。`copy` 拷贝所有元素，`copy_if` 拷贝那些满足 `unaryPred` 的元素，`copy_n` 拷贝前 `n` 个元素。输入序列必须有至少 `n` 个元素。

```
move(beg, end, dest)
```

对输入序列中的每个元素调用 `std::move`（参见 13.6.1 节，第 472 页），将其移动到迭代器 `dest` 开始的序列中。

```
transform(beg, end, dest, unaryOp)
transform(beg, end, beg2, dest, binaryOp)
```

调用给定操作，并将结果写到 `dest` 中。第一个版本对输入范围中每个元素应用一元操作。第二个版本对两个输入序列中的元素应用二元操作。

```
replace_copy(beg, end, dest, old_val, new_val)
replace_copy_if(beg, end, dest, unaryPred, new_val)
```

将每个元素拷贝到 `dest`，将指定的元素替换为 `new_val`。第一个版本替换那些 `==old_val` 的元素。第二个版本替换那些满足 `unaryPred` 的元素。

```
merge(beg1, end1, beg2, end2, dest)
merge(beg1, end1, beg2, end2, dest, comp)
```

两个输入序列必须都是有序的。将合并后的序列写入到 `dest` 中。第一个版本用 `<` 运算符比较元素；第二个版本则使用给定比较操作。

875 使用前向迭代器的写算法

这些算法要求前向迭代器，由于它们向输入序列写入元素，迭代器必须具有写入元素的权限。

```
iter_swap(iter1, iter2)
swap_ranges(beg1, end1, beg2)
```

交换 `iter1` 和 `iter2` 所表示的元素，或将输入范围中所有元素与 `beg2` 开始的第二个序列中所有元素进行交换。两个范围不能有重叠。`iter_swap` 返回 `void`，`swap_ranges` 返回递增后的 `beg2`，指向最后一个交换元素之后的位置。

```
replace(beg, end, old_val, new_val)
replace_if(beg, end, unaryPred, new_val)
```

用 `new_val` 替换每个匹配元素。第一个版本使用 `==` 比较元素与 `old_val`，第二个版本替换那些满足 `unaryPred` 的元素。

使用双向迭代器的写算法

这些算法需要在序列中有反向移动的能力，因此它们要求双向迭代器（`bidirectional iterator`）。

```
copy_backward(beg, end, dest)
```

move_backward(beg, end, dest)

从输入范围中拷贝或移动元素到指定目的位置。与其他算法不同，dest 是输出序列的尾后迭代器（即，目的序列恰在 dest 之前结束）。输入范围中的尾元素被拷贝或移动到目的序列的尾元素，然后是倒数第二个元素被拷贝/移动，依此类推。元素在目的序列中的顺序与在输入序列中相同。如果范围为空，则返回值为 dest；否则，返回值表示从*beg 中拷贝或移动的元素。

inplace_merge(beg, mid, end)

inplace_merge(beg, mid, end, comp)

将同一个序列中的两个有序子序列合并为单一的有序序列。beg 到 mid 间的子序列和 mid 到 end 间的子序列被合并，并被写入到原序列中。第一个版本使用<比较元素，第二个版本使用给定的比较操作，返回 void。

A.2.5 划分与排序算法

对于序列中的元素进行排序，排序和划分算法提供了多种策略。

每个排序和划分算法都提供稳定和不稳定版本（参见 10.3.1 节，第 345 页）。稳定算法保证保持相等元素的相对顺序。由于稳定算法会做更多工作，可能比不稳定版本慢得多并消耗更多内存。

划分算法

◀ 876

一个划分算法将输入范围中的元素划分为两组。第一组包含那些满足给定谓词的元素，第二组则包含不满足谓词的元素。例如，对于一个序列中的元素，我们可以根据元素是否是奇数或者单词是否以大写字母开头等来划分它们。这些算法都要求双向迭代器。

is_partitioned(beg, end, unaryPred)

如果所有满足谓词 unaryPred 的元素都在不满足 unaryPred 的元素之前，则返回 true。若序列为空，也返回 true。

partition_copy(beg, end, dest1, dest2, unaryPred)

将满足 unaryPred 的元素拷贝到 dest1，并将不满足 unaryPred 的元素拷贝到 dest2。返回一个迭代器 pair（11.2.3 节，第 379 页），其 first 成员表示拷贝到 dest1 的元素的末尾，second 表示拷贝到 dest2 的元素的末尾。输入序列与两个目的序列都不能重叠。

partition_point(beg, end, unaryPred)

输入序列必须是已经用 unaryPred 划分过的。返回满足 unaryPred 的范围的尾后迭代器。如果返回的迭代器不是 end，则它指向的元素及其后的元素必须都不满足 unaryPred。

stable_partition(beg, end, unaryPred)

partition(beg, end, unaryPred)

使用 unaryPred 划分输入序列。满足 unaryPred 的元素放置在序列开始，不满足的元素放在序列尾部。返回一个迭代器，指向最后一个满足 unaryPred 的元素之后的位置，如果所有元素都不满足 unaryPred，则返回 beg。

排序算法

这些算法要求随机访问迭代器。每个排序算法都提供两个重载的版本。一个版本用元

素的<运算符来比较元素，另一个版本接受一个额外参数来指定排序关系（11.2.2 节，第 378 页）。`partial_sort_copy` 返回一个指向目的位置的迭代器，其他排序算法都返回 `void`。

`partial_sort` 和 `nth_element` 算法都只进行部分排序工作，它们常用于不需要排序整个序列的场合。由于这些算法工作量更少，它们通常比排序整个输入序列的算法更快。

```
sort(beg, end)
stable_sort(beg, end)
sort(beg, end, comp)
stable_sort(beg, end, comp)
```

排序整个范围。

877 `is_sorted(beg, end)`
`is_sorted(beg, end, comp)`
`is_sorted_until(beg, end)`
`is_sorted_until(beg, end, comp)`

`is_sorted` 返回一个 `bool` 值，指出整个输入序列是否有序。`is_sorted_until` 在输入序列中查找最长初始有序子序列，并返回子序列的尾后迭代器。

```
partial_sort(beg, mid, end)
partial_sort(beg, mid, end, comp)
```

排序 `mid-beg` 个元素。即，如果 `mid-beg` 等于 42，则此函数将值最小的 42 个元素有序放在序列前 42 个位置。当 `partial_sort` 完成后，从 `beg` 开始直至 `mid` 之前的范围内的元素就都已排好序了。已排序范围中的元素都不会比 `mid` 后的元素更大。未排序区域中元素的顺序是未指定的。

```
partial_sort_copy(beg, end, destBeg, destEnd)
partial_sort_copy(beg, end, destBeg, destEnd, comp)
```

排序输入范围中的元素，并将足够多的已排序元素放到 `destBeg` 和 `destEnd` 所指示的序列中。如果目的范围的大小大于等于输入范围，则排序整个输入序列并存入从 `destBeg` 开始的范围。如果目的范围大小小于输入范围，则只拷贝输入序列中与目的范围一样多的元素。

算法返回一个迭代器，指向目的范围中已排序部分的尾后迭代器。如果目的序列的大小小于或等于输入范围，则返回 `destEnd`。

```
nth_element(beg, nth, end)
nth_element(beg, nth, end, comp)
```

参数 `nth` 必须是一个迭代器，指向输入序列中的一个元素。执行 `nth_element` 后，此迭代器指向的元素恰好是整个序列排好序后此位置上的值。序列中的元素会围绕 `nth` 进行划分：`nth` 之前的元素都小于等于它，而之后的元素都大于等于它。

A.2.6 通用重排操作

这些算法重排输入序列中元素的顺序。前两个算法 `remove` 和 `unique`，会重排序列，使得排在序列第一部分的元素满足某种标准。它们返回一个迭代器，标记子序列的末尾。其他算法，如 `reverse`、`rotate` 和 `random_shuffle` 都重排整个序列。

这些算法的基本版本都进行“原址”操作，即，在输入序列自身内部重排元素。三个

重排算法提供“拷贝”版本。这些_copy版本完成相同的重排工作，但将重排后的元素写入到一个指定目的序列中，而不是改变输入序列。这些算法要求输出迭代器来表示目的序列。

使用前向迭代器的重排算法

878

这些算法重排输入序列。它们要求迭代器至少是前向迭代器。

```
remove(beg, end, val)
remove_if(beg, end, unaryPred)
remove_copy(beg, end, dest, val)
remove_copy_if(beg, end, dest, unaryPred)
```

从序列中“删除”元素，采用的办法是用保留的元素覆盖要删除的元素。被删除的是那些==val 或满足 unaryPred 的元素。算法返回一个迭代器，指向最后一个删除元素的尾后位置。

```
unique(beg, end)
unique(beg, end, binaryPred)
unique_copy(beg, end, dest)
unique_copy_if(beg, end, dest, binaryPred)
```

重排序列，对相邻的重复元素，通过覆盖它们来进行“删除”。返回一个迭代器，指向不重复元素的尾后位置。第一个版本用==确定两个元素是否相同，第二个版本使用谓词检测相邻元素。

```
rotate(beg, mid, end)
rotate_copy(beg, mid, end, dest)
```

围绕 mid 指向的元素进行元素转动。元素 mid 成为首元素，随后是 mid+1 到 end 之前的元素，再接着是 beg 到 mid 之前的元素。返回一个迭代器，指向原来在 beg 位置的元素。

使用双向迭代器的重排算法

由于这些算法要反向处理输入序列，它们要求双向迭代器。

```
reverse(beg, end)
reverse_copy(beg, end, dest)
```

翻转序列中的元素。reverse 返回 void，reverse_copy 返回一个迭代器，指向拷贝到目的序列的元素的尾后位置。

使用随机访问迭代器的重排算法

由于这些算法要随机重排元素，它们要求随机访问迭代器。

```
random_shuffle(beg, end)
random_shuffle(beg, end, rand)
shuffle(beg, end, Uniform_rand)
```

混洗输入序列中的元素。第二个版本接受一个可调对象参数，该对象必须接受一个正整数值，并生成 0 到此值的包含区间内的一个服从均匀分布的随机整数。shuffle 的第三个参数必须满足均匀分布随机数生成器的要求（参见 17.4 节，第 659 页）。所有版本都返回 void。

879

A.2.7 排列算法

排列算法生成序列的字典序排列。对于一个给定序列，这些算法通过重排它的一个排列来生成字典序中下一个或前一个排列。算法返回一个 `bool` 值，指出是否还有下一个或前一个排列。

为了理解什么是下一个或前一个排列，考虑下面这个三字符的序列：abc。它有六种可能的排列：abc、acb、bac、bca、cab 及 cba。这些排列是按字典序递增序列出的。即，abc 是第一个排列，这是因为它的第一个元素小于或等于任何其他排列的首元素，并且它的第二个元素小于任何其他首元素相同的排列。类似的，acb 排在下一位，原因是它以 a 开头，小于任何剩余排列的首元素。同理，以 b 开头的排列也都排在以 c 开头的排列之前。

对于任意给定的排列，基于单个元素的一个特定的序，我们可以获得它的前一个和下一个排列。给定排列 bca，我们知道其前一个排列为 bac，下一个排列为 cab。序列 abc 没有前一个排列，而 cba 没有下一个排列。

这些算法假定序列中的元素都是唯一的，即，没有两个元素的值是一样的。

为了生成排列，必须既向前又向后处理序列，因此算法要求双向迭代器。

```
is_permutation(beg1, end1, beg2)
is_permutation(beg1, end1, beg2, binaryPred)
```

如果第二个序列的某个排列和第一个序列具有相同数目的元素，且元素都相等，则返回 `true`。第一个版本用 `==` 比较元素，第二个版本使用给定的 `binaryPred`。

```
next_permutation(beg, end)
next_permutation(beg, end, comp)
```

如果序列已经是最后一个排列，则 `next_permutation` 将序列重排为最小的排列，并返回 `false`。否则，它将输入序列转换为字典序中下一个排列，并返回 `true`。第一个版本使用元素的 `<` 运算符比较元素，第二个版本使用给定的比较操作。

```
prev_permutation(beg, end)
prev_permutation(beg, end, comp)
```

类似 `next_permutation`，但将序列转换为前一个排列。如果序列已经是最小的排列，则将其重排为最大的排列，并返回 `false`。

880 A.2.8 有序序列的集合算法

集合算法实现了有序序列上的一般集合操作。这些算法与标准库 `set` 容器不同，不要与 `set` 上的操作相混淆。这些算法提供了普通顺序容器（`vector`、`list` 等）或其他序列（如输入流）上的类集合行为。

这些算法顺序处理元素，因此要求输入迭代器。他们还接受一个表示目的序列的输出迭代器，唯一的例外是 `includes`。这些算法返回递增后的 `dest` 迭代器，表示写入 `dest` 的最后一个元素之后的位置。

每种算法都有重载版本，第一个使用元素类型的 `<` 运算符，第二个使用给定的比较操作。

```
includes(beg, end, beg2, end2)
includes(beg, end, beg2, end2, comp)
```

如果第二个序列中每个元素都包含在输入序列中，则返回 true。否则返回 false。

```
set_union(beg, end, beg2, end2, dest)
set_union(beg, end, beg2, end2, dest, comp)
```

对两个序列中的所有元素，创建它们的有序序列。两个序列都包含的元素在输出序列中只出现一次。输出序列保存在 dest 中。

```
set_intersection(beg, end, beg2, end2, dest)
set_intersection(beg, end, beg2, end2, dest, comp)
```

对两个序列都包含的元素创建一个有序序列。结果序列保存在 dest 中。

```
set_difference(beg, end, beg2, end2, dest)
set_difference(beg, end, beg2, end2, dest, comp)
```

对出现在第一个序列中，但不在第二个序列中的元素，创建一个有序序列。

```
set_symmetric_difference(beg, end, beg2, end2, dest)
set_symmetric_difference(beg, end, beg2, end2, dest, comp)
```

对只出现在一个序列中的元素，创建一个有序序列。

A.2.9 最小值和最大值

这些算法使用元素类型的<运算符或给定的比较操作。第一组算法对值而非序列进行操作。第二组算法接受一个序列，它们要求输入迭代器。

```
min(val1, val2)
min(val1, val2, comp)
min(init_list)
min(init_list, comp)
max(val1, val2)
max(val1, val2, comp)
max(init_list)
max(init_list, comp)
```

881

返回 val1 和 val2 中的最小值/最大值，或 initializer_list 中的最小值/最大值。两个实参的类型必须完全一致。参数和返回类型都是 const 的引用，意味着对象不会被拷贝。

```
minmax(val1, val2)
minmax(val1, val2, comp)
minmax(init_list)
minmax(init_list, comp)
```

返回一个 pair(参见 11.2.3 节, 第 379 页), 其 first 成员为提供的值中的较小者, second 成员为较大者。initializer_list 版本返回一个 pair, 其 first 成员为 list 中的最小值, second 为最大值。

```
min_element(beg, end)
min_element(beg, end, comp)
max_element(beg, end)
max_element(beg, end, comp)
minmax_element(beg, end)
minmax_element(beg, end, comp)
```

`min_element` 和 `max_element` 分别返回指向输入序列中最小和最大元素的迭代器。`minmax_element` 返回一个 `pair`，其 `first` 成员为最小元素，`second` 成员为最大元素。

字典序比较

此算法比较两个序列，根据第一对不相等的元素的相对大小来返回结果。算法使用元素类型的 `<` 运算符或给定的比较操作。两个序列都要求用输入迭代器给出。

```
lexicographical_compare(beg1, end1, beg2, end2)
lexicographical_compare(beg1, end1, beg2, end2, comp)
```

如果第一个序列在字典序中小于第二个序列，则返回 `true`。否则，返回 `false`。如果一个序列比另一个短，且所有元素都与较长序列的对应元素相等，则较短序列在字典序中更小。如果序列长度相等，且对应元素都相等，则在字典序中任何一个都不大于另外一个。

A.2.10 数值算法

数值算法定义在头文件 `numeric` 中。这些算法要求输入迭代器；如果算法输出数据，则使用输出迭代器表示目的位置。

882

```
accumulate(beg, end, init)
accumulate(beg, end, init, binaryOp)
```

返回输入序列中所有值的和。和的初值从 `init` 指定的值开始。返回类型与 `init` 的类型相同。第一个版本使用元素类型的 `+` 运算符，第二个版本使用指定的二元操作。

```
inner_product(beg1, end1, beg2, init)
inner_product(beg1, end1, beg2, init, binOp1, binOp2)
```

返回两个序列的内积，即，对应元素的积的和。两个序列一起处理，来自两个序列的元素相乘，乘积被累加起来。和的初值由 `init` 指定，`init` 的类型确定了返回类型。

第一个版本使用元素类型的乘法 (`*`) 和加法 (`+`) 运算符。第二个版本使用给定的二元操作，使用第一个操作代替加法，第二个操作代替乘法。

```
partial_sum(beg, end, dest)
partial_sum(beg, end, dest, binaryOp)
```

将新序列写入 `dest`，每个新元素的值都等于输入范围中当前位置和之前位置上所有元素之和。第一个版本使用元素类型的 `+` 运算符；第二个版本使用指定的二元操作。算法返回递增后的 `dest` 迭代器，指向最后一个写入元素之后的位置。

```
adjacent_difference(beg, end, dest)
adjacent_difference(beg, end, dest, binaryOp)
```

将新序列写入 `dest`，每个新元素（除了首元素之外）的值都等于输入范围中当前位置和前一个位置元素之差。第一个版本使用元素类型的 `-` 运算符，第二个版本使用指定的二元操作。

```
iota(beg, end, val)
```

将 `val` 赋予首元素并递增 `val`。将递增后的值赋予下一个元素，继续递增 `val`，然后将递增后的值赋予序列中的下一个元素。继续递增 `val` 并将其新值赋予输入序列中的后续元素。

A.3 随机数

标准库定义了一组随机数引擎类和适配器，使用不同数学方法生成伪随机数。标准库还定义了一组分布模板，根据不同的概率分布生成随机数。引擎和分布类型的名字都与它们的数学性质相对应。

这些类如何生成随机数的细节已经大大超出了本书的范围。在本节中，我们将列出这些引擎和分布类型，但读者需要查询其他资料来学习如何使用这些类型。

A.3.1 随机数分布

883

除了总是生成 `bool` 类型的 `bernoulli_distribution` 外，其他分布类型都是模板。每个模板都接受单个类型参数，它指出了分布生成的结果类型。

分布类与我们已经用过的其他类模板不同，它们限制了我们可以为模板类型指定哪些类型。一些分布模板只能用来生成浮点数，而其他模板只能用来生成整数。

在下面的描述中，我们通过将类型说明为 `template_name<RealT>` 来指出分布生成浮点数。对这些模板，我们可以用 `float`、`double` 或 `long double` 代替 `RealT`。类似的，`IntT` 表示要求一个内置整型类型，但不包括 `bool` 类型或任何 `char` 类型。可以用来代替 `IntT` 的类型是 `short`、`int`、`long`、`long long`、`unsigned short`、`unsigned int`、`unsigned long` 或 `unsigned long long`。

分布模板定义了一个默认模板类型参数（参见 17.4.2 节，第 664 页）。整型分布的默认参数是 `int`，生成浮点数的模板的默认参数是 `double`。

每个分布的构造函数都有这种分布特定的参数。某些参数指出了分布的范围。这些范围与迭代器范围不同，都是包含的。

均匀分布

```
uniform_int_distribution<IntT> u(m, n);
uniform_real_distribution<RealT> u(x, y);
```

生成指定类型的，在给定包含范围内的值。`m`（或 `x`）是可以返回的最小值；`n`（或 `y`）是最大值。`m` 默认为 0；`n` 默认为类型 `IntT` 对象可以表示的最大值。`x` 默认为 0.0，`y` 默认为 1.0。

伯努利分布

```
bernoulli_distribution b(p);
```

以给定概率 `p` 生成 `true`；`p` 的默认值为 0.5。

```
binomial_distribution<IntT> b(t, p);
```

分布是按采样大小为整型值 `t`，概率为 `p` 生成的；`t` 的默认值为 1，`p` 的默认值为 0.5。

```
geometric_distribution<IntT> g(p);
```

每次试验成功的概率为 `p`；`p` 的默认值为 0.5。

```
negative_binomial_distribution<IntT> nb(k, p);
```

`k`（整型值）次试验成功的概率为 `p`；`k` 的默认值为 1，`p` 的默认值为 0.5。